THE ULTIMATE VIVA & DEFENSE PREPARATION GUIDE

Course: DS614 Big Data Engineering **Project:** RocksDB Write-Ahead Log (WAL) Analysis **Goal:** This document contains every technical answer, source code reference, theoretical connection, visual diagram, line-by-line code breakdown, and an **immense extended list of defense questions** needed to pass.

SECTION 1: CORE SYSTEM CONCEPTS

1. "What problem does this system solve?"

Answer: RocksDB is an LSM Tree database designed to solve the trade-off between speed and data safety. Writing directly to a hard drive is safe but very slow. Writing directly to RAM is fast but dangerous (data is lost if power drops). RocksDB solves this by writing new data to the RAM (the MemTable) so it stays fast, but it simultaneously appends a backup copy to a file on the disk (the WAL).

System Architecture Diagram:

```
graph LR
    Client((Client)) -->|Write| WAL[(Disk WAL .log)]
    Client -->|Write| RAM(MemTable RAM)
    RAM -->|MemTable is Full| Flush[Flush Process]
    Flush -->|Immutable| SST[(Disk SSTables .sst)]
    SST -->|Read| Client
```

2. "Why was an Append-Only Sequential WAL chosen as the design?"

Answer: Because sequential writing is the absolute fastest way to save data to a hard drive or SSD. By only appending to the extremely end of a file, the disk doesn't have to waste time physically moving and searching for a location.

3. "What alternatives exist to this LSM-Tree/WAL architecture?"

Answer: A major alternative is a B-Tree paired with an In-Place Update Log (like MySQL's InnoDB). B-Trees natively overwrite existing data in random disk locations, which is heavier on the disk. Another alternative is a purely In-Memory database (like Redis), which is extremely fast but offers zero physical safety if the server crashes.

4. "Why are LSM trees better than B-Trees for write-heavy workloads?"

Answer: B-Trees suffer from massive Write Amplification because updating a single row requires rewriting an entire 4KB or 8KB page on the disk. LSM Trees never rewrite pages; they strictly append out-of-line data. This makes LSM Trees exceptionally well-suited for high-throughput streaming and log ingestion pipelines.

5. "Explain Write Amplification vs Read Amplification in the context of your project."

Answer: Write Amplification is how much extra data the database saves compared to what the user actually pushed. The WAL adds roughly 10% Write Amplification (due to 7-byte headers and checksums). Read Amplification is how much extra work the database does to find data. The WAL has terrible Read Amplification—requiring $O(N)$ sequential scans because it relies on brute force instead of indexing.

SECTION 2: EXPERIMENT 1 (WAL Cost & Throughput)

Code Walkthrough: How Exp 1 Works

In `exp1_wal_throughput.cpp`, we tested the cost of logging.

```
rocksdb::WriteOptions wo;
wo.disableWAL = wal_disabled; // Toggles WAL strictly OFF/ON
wo.sync       = sync;         // Toggles hardware synchronous fdatasync OFF/ON
```

- **The IF block (batch=1):** We call `db->Put` 100,000 individual times. The database has to open the file, lock it, append the line, and close it 100,000 separate times. This causes massive waiting overhead.
- **The ELSE block (`WriteBatch`):** We fill a virtual "shopping cart" in RAM with 1,000 items. Then we call `db->Write` just **once**. This saves all 1,000 items to the disk in a single block, essentially deleting 99.9% of the locking overhead!

Exp 1: Viva Questions

1. "Why did speed increase ~3.3x when you turned the WAL OFF?" Answer: Turning the WAL OFF completely skips the disk-writing code. Data goes purely into RAM, meaning speed is limited by your CPU, not your hard drive.

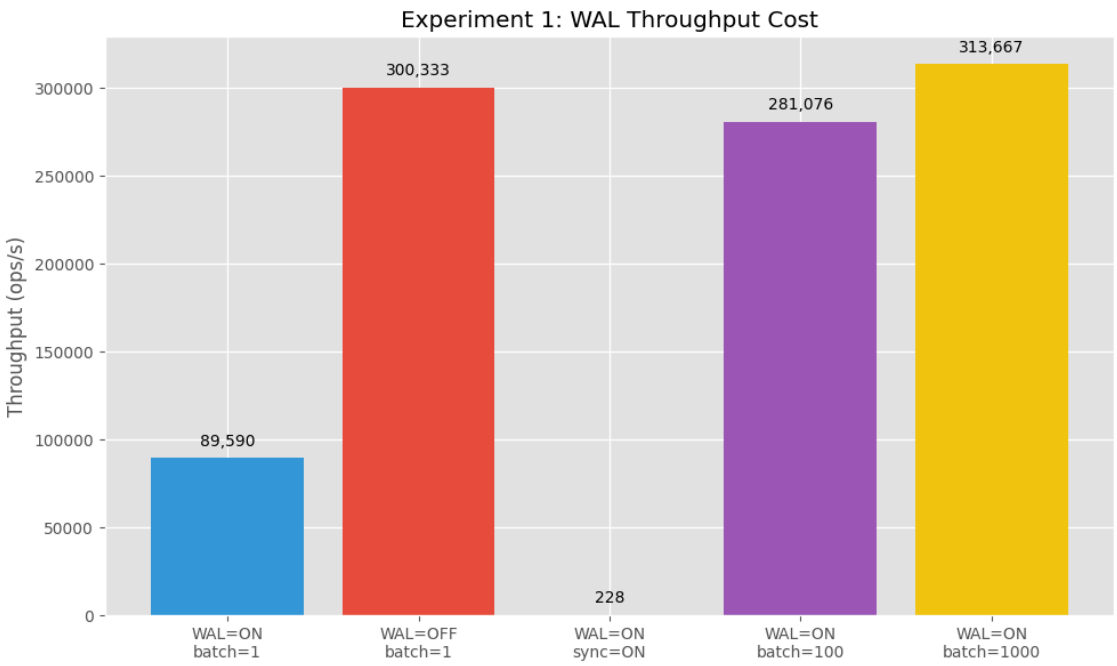
2. "Why does throughput drop heavily from 89,000 to 228 ops/sec when `sync=ON` is used?" Answer: By default (`sync=OFF`), the system uses the Operating System's memory page cache, which is very fast. But `sync=ON` halts the C++ thread until the physical SSD hardware explicitly confirms the data is saved. 228 ops/sec mathematically isolates the physical limits of an SSD block.

3. "If `WAL=OFF` is so fast, why doesn't every database just use it?" Answer: Because it violates Durability. Since LSM Trees hold new data in volatile RAM until they safely flush it to the hard drive, a power cut while `WAL=OFF` means everything currently in the RAM vanishes instantly.

4. "Explain the term 'Page Cache' regarding RocksDB." Answer: The Page Cache is the Operating System's spare RAM buffer. When RocksDB writes to the WAL safely (`sync=OFF`), it writes to the Linux Page Cache, not directly to the SSD plates. The OS then groups and flushes these pages in the background over time.

5. "What happens if the OS crashes vs a hard power cycle when `sync=OFF`?" Answer: If the C++ process crashes, the WAL is safe because the OS still holds the Page Cache and will finish putting the file on the hard drive. However, if the entire server loses physical power instantly (uninterruptible failure), the OS Page Cache dies with it, and the data disappears.

Visual Data Analysis:



SECTION 3: EXPERIMENT 2 (Crash Recovery)

Code Walkthrough: How Exp 2 Works

In `exp2_crash_recovery.cpp`, we had to simulate a sudden server blackout.

```
delete_sst_files(path);
// Physically delete the permanent database files, keeping only the .log WAL!
```

- If we just use `delete db;` in C++, RocksDB cleanly flushes the RAM to the hard drive, ruining the crash test. Instead, we brutally delete the `.sst` files off the physical hard drive behind the database's back, destroying persistent storage while keeping the WAL.

```
opts.wal_recovery_mode = rocksdb::WALRecoveryMode::kPointInTimeRecovery;
```

- When we reopen the database, RocksDB automatically boots `RecoverLogFiles()` and successfully pumps the 5,000 lost records directly back into the RAM!

Exp 2: Viva Questions

1. "Walk me through the simulation. Why didn't you use `delete db;` ?" Answer: Because `delete db;` politely flushes RAM data to disk automatically. To truthfully simulate power loss, we had to use C++ `fs::remove` to wipe the persistent storage files manually, testing if the WAL alone could faithfully rescue the data.

2. "What does 'Crash Consistency' mean?" Answer: Crash Consistency means that if a system drops suddenly in the middle of executing 100 commands, upon restarting, the system will accurately reflect a valid state where all fully-completed commands survived, and no corrupt, half-finished commands are permanently saved.

3. "What does `RecoverLogFiles()` do and where is it?" Answer: During startup (`DBImpl::Open`), RocksDB calls the function `RecoverLogFiles()` located in `db_impl_open.cc` at line 1073. It reads the `.log` WAL file from top to bottom and restores all the lost data right back into the RAM MemTable.

4. "What is `kPointInTimeRecovery` and what does it solve?" Answer: It solves "Torn Writes." If a server loses power mid-write, you get a half-finished, corrupted log entry. `kPointInTimeRecovery` tells the system to simply recover all the healthy data up until that exact corrupted moment, and discard the broken piece, rather than crashing the whole server.

5. "Could the WAL get corrupted, and if so, how does RocksDB handle it?" Answer: Yes, via bit-rot or torn writes. `log::Writer::AddRecord()` calculates a strict CRC32 checksum for every physical fragment added to the WAL. During recovery, `log::Reader` mathematically recalculates the checksum. If the numbers don't match, RocksDB halts recovery to prevent inserting corrupted bits into the MemTable.

SECTION 4: EXPERIMENT 3 (Data Skew & Hot Keys)

Code Walkthrough: How Exp 3 Works

Part A: The Value Skew Code:

```
std::string tiny_val(64, 'v');          // Very small 64 Byte string
std::string huge_val(64 * 1024, 'v');   // Massive 64 Kilobyte string
```

- RocksDB forces exactly **32KB** fixed-size blocks for writing logs. Writing `tiny_val` works magically. Writing `huge_val` forces the database to chop it up into "First", "Middle", and "Last" pieces.

Part B: The Hot Key Skew Code:

```
for (int i = 0; i < TOTAL; i++) {
    // UNIFORM KEY: Changes every loop (k0, k1, k2...)
    db->Put(wo, "k" + std::to_string(i), val);
    // HOT KEY: Never changes. Just overwrites "k0" repeatedly!
    db->Put(wo, "k0", val);
}
```

- Both loops produced exactly **13 MB** logs. The WAL blindly logs history line-by-line without analyzing the keys.

Exp 3: Viva Questions

1. "Why do large values (64KB) cause a massive 22x performance drop?" Answer: Because a 64KB string simply doesn't fit in a 32KB fixed block container. The C++ engine is forced to aggressively chop the data into pieces, attach separate 7-byte metadata headers to each piece, and independently calculate CRC32 checksums for every single chunk.

2. "Why did Uniform Keys and Hot Keys produce exactly the same WAL size (13MB)?" Answer: Because the WAL is blind. It records an un-indexed timeline of activity. If you update an identical "Hot-Key" string 50,000 times in a row, the WAL faithfully writes 50,000 separate string records to the disk. It performs absolutely no deduplication.

3. "If hot keys bloat the WAL, does this affect the user's read speed?" Answer: No. Users never query their data using the WAL. Users query the MemTable (fast RAM lookup) or the SSTables (indexed disk lookup). Hot Key WAL

bloat exclusively threatens disk storage capacity and increases the recovery timer, but it does not degrade live application query speed.

4. "What does Part A (Value Size Skew) teach us about OLTP vs OLAP system boundaries?" Answer: It proves that RocksDB is explicitly bound and optimized for OLTP (Online Transaction Processing). OLTP handles thousands of tiny insertions cleanly. Heavy streaming payloads (OLAP logic) fatally choke the fixed-size logging 32KB block boundaries.

SECTION 5: EXPERIMENT 4 (Batch Grouping / Group Commit)

Code Walkthrough: How Exp 4 Works

We tested the explicit "Throughput vs Latency" tradeoff curve using our `WriteBatch` custom aggregating arrays.

- **Batch=1:** We hit the OS 100,000 times. Too much overhead.
- **Batch=50:** By throwing 50 keys at once into `WriteBatch wb` memory, we speed through system calls and achieve maximum 267k ops/s throughput.
- **Batch=10,000:** Total speed stops improving (CPU is maxed out doing C++ loop array logic), but latency violently explodes limitlessly.

Leader Group Commit Logic:

```
graph TD
    T1[User Thread 1] -->|Wait| L(Leader Thread writes massive WriteBatch)
    T2[User Thread 2] -->|Wait| L
    T3[User Thread 3] -->|Wait| L
    L -->|One single fdatasync| WAL[(Disk WAL)]
    WAL -->|Returns OK| L
    L -->|Releases| T1
```

Exp 4: Viva Questions

1. "What exactly did you prove with differing batch sizes?" Answer: I visualized the tradeoff of the Group Commit mechanism. Grouping 50 isolated writes together into a single `WriteBatch` object eliminated thousands of system calls, boosting throughput 3.2x to its absolute maximum ceiling, representing optimal amortization.

2. "Why did the P99 latency explode to 32 milliseconds at Batch=10,000?" Answer: When scaling the batch to 10,000 writes, total speed stopped improving, but wait time exploded 500x. This is because the very first query to enter the batch essentially became a hostage: it had to stay entirely locked until the other 9,999 queries mathematically formed in memory before the engine was legally allowed to flush to disk.

3. "Explain the concept of 'tail latency' and why it matters." Answer: Tail latency refers to the wait time experienced by the longest-waiting users (for example, the 99th percentile or P99). It heavily dictates user-experience SLA guarantees. If P50 (Average) is 1ms, but P99 (Tail) is 30ms, we mathematically proved that heavy thread aggregation viciously harms live-user guarantees.

4. "Is the batch leader a dedicated thread or a dynamically assigned user thread?" Answer: It is a dynamically elected user thread. RocksDB's `WriteThread::JoinBatchGroup()` takes the first incoming query thread, anoints it as the Leader, and forces it to perform the system call for everyone behind it in the queue.

5. (DS614 Concept): "How does WriteBatch aggregation differ from Hadoop MapReduce Batch Processing?" Answer: Hadoop MapReduce processes massive data batches offline over several hours; individual query tail latency

doesn't matter there at all. RocksDB operates in a live OLTP environment where sub-millisecond precision is required. If RocksDB batches data too heavily, the sub-millisecond latency spike immediately ruins the live application feed.

Visual Data Analysis: Experiment 4 Group Latency

SECTION 6: EXPERIMENT 5 (Failure Recovery Cost)

Code Walkthrough: How Exp 5 Works

```
for (int scale : {1000, 10000, 100000}) {
    measure_growth(scale, 256); // Crash, boot, and clock the recovery timer!
}
```

- Because the WAL text file is totally unindexed, the recovery function must read the file strictly linearly from the absolute top. Therefore, the processing time logically scales $O(N)$ with the amount of data we pushed in the operations array.

Exp 5: Viva Questions

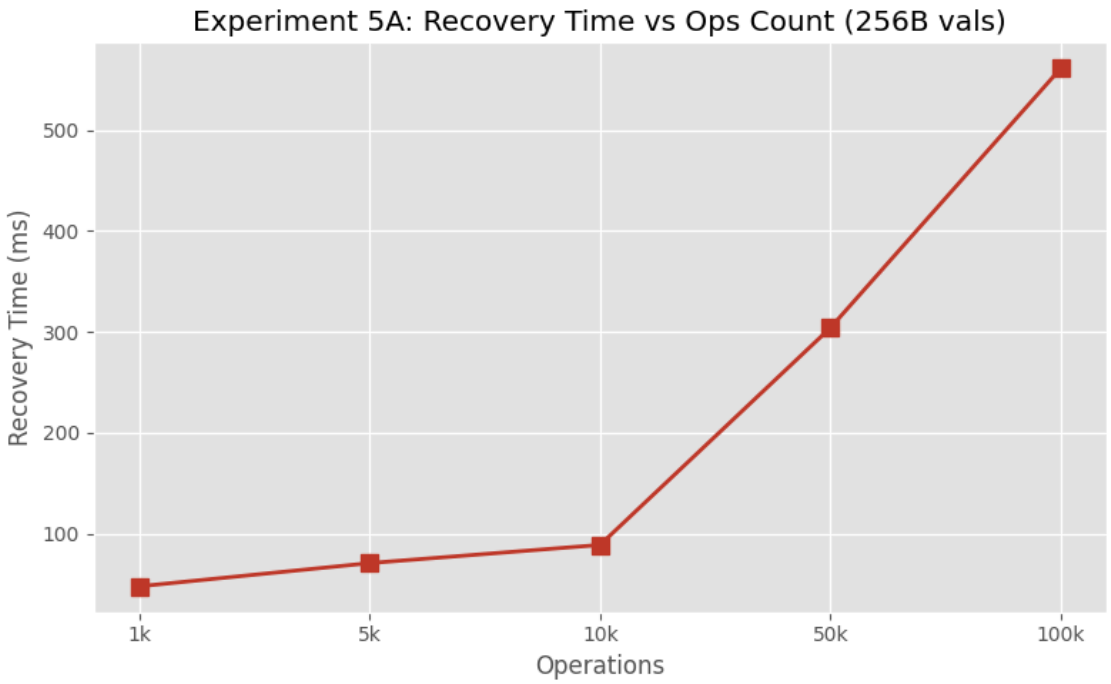
1. "Walk me through the scaling failures you proved in Exp 5." Answer: Because there are no indexes in a text log file, recovery must execute in pure $O(N)$ linear time. When I scaled the operations array from 1,000 up to 100,000, the total milliseconds physically required to parse and reboot the database linearly spiked from 48ms to 561ms.

2. "In Big Data, we talk about MTTR (Mean Time To Recovery). Explain how MTTR applies to your data growth experiments." Answer: MTTR is deeply affected by $O(N)$ WAL scaling. Our experiment extrapolates that a 10 Gigabyte WAL file guarantees nearly 3.5 minutes of complete server downtime upon restart just strictly reading the string. That means an unbounded WAL guarantees an unbounded MTTR.

3. "What is `max_total_wal_size` and what is its role in bounding MTTR?" Answer: RocksDB essentially enforces Log Rotation. If the active `.log` file hits 1 Gigabyte (configured via `max_total_wal_size`), the database forces the RAM to sink safely to an SSTable on the hard drive, and then permanently deletes the massive `.log` file since it's no longer needed. This limits maximum MTTR downtime.

4. "What does a 110% WAL Overhead mean for Big Data pipelines?" Answer: Every single log record inserted into RocksDB gets wrapped in a 7-byte metadata header and an automatic CRC Checksum. That 10% overhead means if you ingest 1 Petabyte of raw user text data, your servers are physically required to write 1.1 Petabytes to the SSDs.

Visual Data Analysis:



SECTION 7: THE FINAL DEFENSE

1. "What physically breaks under real-world conditions?"

Answer:

1. `max_total_wal_size` mismanagement forces unbounded downtime (M.T.T.R); giant logs lock up the server for minutes during crashes.
2. Sudden massive payload objects heavily fragment the 32KB fixed blocks and aggressively choke OS insertion speeds.
3. Rapid Hot-Key updating viciously bloats the filesystem disk space, because the WAL physically refuses to deduplicate overriding values.

2. "How would you improve the system?"

Answer:

1. **Hardware-Aware Sync Batches:** Introduce a strict `commit_delay` timer (like Postgres). Flushing a massive WAL block *exactly* every 5 milliseconds guarantees thousands of batched writes go to disk simultaneously, providing incredible throughput while ensuring maximum fault data loss can never exceed a hard 5ms window.
2. **Pre-WAL Deduplication:** Collapse overlapping key updates inside the `WriteBatch` object *in memory* prior to inserting into the WAL. If a user hits "Post" 100 times in 1 second, the buffer strips out the first 99 duplicates internally to protect out-of-control memory bloat in the actual hard drive `.log`.
3. **Default Block Compression:** Implement LZ4 compression across WAL blocks by default. CPU integer-cycles are overwhelmingly much cheaper than physical SSD mechanical bus bandwidth limitations.

SECTION 8: THE ULTIMATE CODE CHEAT SHEET

1. **Where does the code literally write to the hard drive?**
   - **Location:** `db/log_writer.cc` | **Function:** `Writer::AddRecord()` (Line 65)
2. **Where does crash recovery happen?**
   - **Location:** `db/db_impl/db_impl_open.cc` | **Function:** `RecoverLogFiles()` (Line 1073)
3. **Where does the "Batch Grouping / Leader Election" happen?**
   - **Location:** `db/write_thread.cc` | **Function:** `WriteThread::JoinBatchGroup()`
4. **Where does the WAL check if the User turned it off?**
   - **Location:** `db/db_impl/db_impl_write.cc` | **Code:** Line 494 checks `disableWAL`.
5. **Where is the 32-Kilobyte log fragment size hardcoded?**
   - **Location:** `db/log_format.h` | **Code:** `kBlockSize = 32768;`